Instituto Tecnológico Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

MÉTODOS NUMÉRICOS

VAZQUES HERRERA VICTOR HUGO

UNIDAD 5
TAREA 1

PRESENTA

257O00081
Brayan Mejia Mora

09 DE NOVIEMBRE DEL 2026

Práctica: Interpolación de Lagrange vs. Diferencias Divididas de Newton

Repositorio de GitHub

Los siguientes links corresponde al repositorio público personal de GitHub:

Interpolación de Lagrange: https://github.com/brianlewyn/numerical-methods-tec-3th-semester/tree/main/Unidad5/InterpolacionLagrange

Diferencias Divididas de Newton: https://github.com/brianlewyn/numerical-methods-tec-3th-semester/tree/main/Unidad5/InterpolacionNewton

Introducción

La interpolación polinomial es una técnica fundamental del análisis numérico que consiste en construir un polinomio $P(x)$ de grado menor o igual a n que pase exactamente por un conjunto de $n+1$ puntos conocidos $(x_0,y_0),(x_1,y_1),\ldots,(x_n,y_n)$. Su importancia radica en que permite aproximar funciones complejas, estimar valores intermedios a partir de datos tabulados y modelar fenómenos experimentales.

Desde el punto de vista algebraico, el problema de interpolación tiene solución única: dados n+1 nodos distintos, existe un único polinomio de grado ≤n que interpola dichos puntos. Sin embargo, existen diversas formas de expresar y calcular dicho polinomio. Dos de las más importantes son la forma de **Lagrange** y la forma de **Newton** con diferencias divididas.

Marco Teórico

Método de los Polinomios de Lagrange

La forma de Lagrange construye el polinomio interpolador como una combinación lineal de polinomios base $L_k(x)$, cada uno de los cuales vale 1 en el nodo x_k y 0 en todos los demás nodos. La expresión general es:

$$P(x) = \sum_{k=0}^{n} y_k \, L_k(x)$$

donde cada polinomio base se define como:

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}$$

Esta formulación tiene la ventaja de ser conceptualmente directa y simétrica, pues no requiere un orden particular de los nodos. Sin embargo, presenta un inconveniente computacional importante: si se desea añadir un nuevo nodo al conjunto de datos, es necesario recalcular todos los polinomios base desde cero, ya que cada $L_k(x)$ depende del conjunto completo de nodos.

Método de Diferencias Divididas de Newton

La forma de Newton expresa el polinomio interpolador de manera jerárquica:

$$P(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + \dots + f[x_0,\dots,x_n](x-x_0)\dots(x-x_{n-1})$$

Los coeficientes $f[x_0,\dots,x_n]$ se denominan diferencias divididas y se calculan recursivamente mediante la tabla de diferencias divididas:

$$f[x_i] = y_i$$

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

La principal ventaja de esta forma es su **carácter incremental**: si se añade un nuevo nodo x_{n+1}, los coeficientes anteriores $f[x_0]$, $f[x_0, x_1]$, …, $f[x_0, \dots, x_n]$ permanecen invariantes, y solo es necesario calcular las nuevas diferencias divididas que involucran al nuevo nodo. Esto convierte a Newton en el método preferido cuando el conjunto de datos puede crecer.

Desarrollo del Código

Corrección en InterpolacionLagrange.java

En la implementación típica del método de Lagrange, el error más común se encuentra en la **condición de exclusión del índice** dentro del bucle interno. La fórmula correcta requiere que el producto omita el término j=k, es decir, que el denominador nunca incluya $(x_k - x_k) = 0$.

```java
public static double interpolacionLagrange(double[] x, double[] y, double xEval) {
    validarDatos(x, y);

    double resultado = 0.0;
    int n = x.length;

    for (int i = 0; i < n; i++) {
        double termino = y[i];

        for (int j = 0; j < n; j++) {
            if (i != j) {
                // Li(xi) = numerador(x) / denominador(xi)
                // numerador(x): (x - x0)(x - x1) ... (x - xn)
                // denomiador(xi): (xi - x0)(xi - x1) ... (x - xn)
                termino *= (xEval - x[j]) / (x[i] - x[j]);
            }
        }

        // termino(xi): y0*L0(x0) + y1*L1(x1) + yi*Li(xi)
        resultado += termino;
    }

    return resultado;
}
```

Justificación matemática

El polinomio base de Lagrange $L_k(x)$ se define como el producto de todos los factores $(x - x_j)/(x_k - x_j)$ **excepto** cuando j=k. Si se incluyera j=k, el denominador contendría el factor $(x_k - x_k) = 0$, lo que produciría una división por cero y, además, el numerador también se anularía, dejando el término indeterminado. La condición j != k garantiza que se respete exactamente la definición matemática del polinomio base.

Pruebas Numéricas y Resultados en Consola

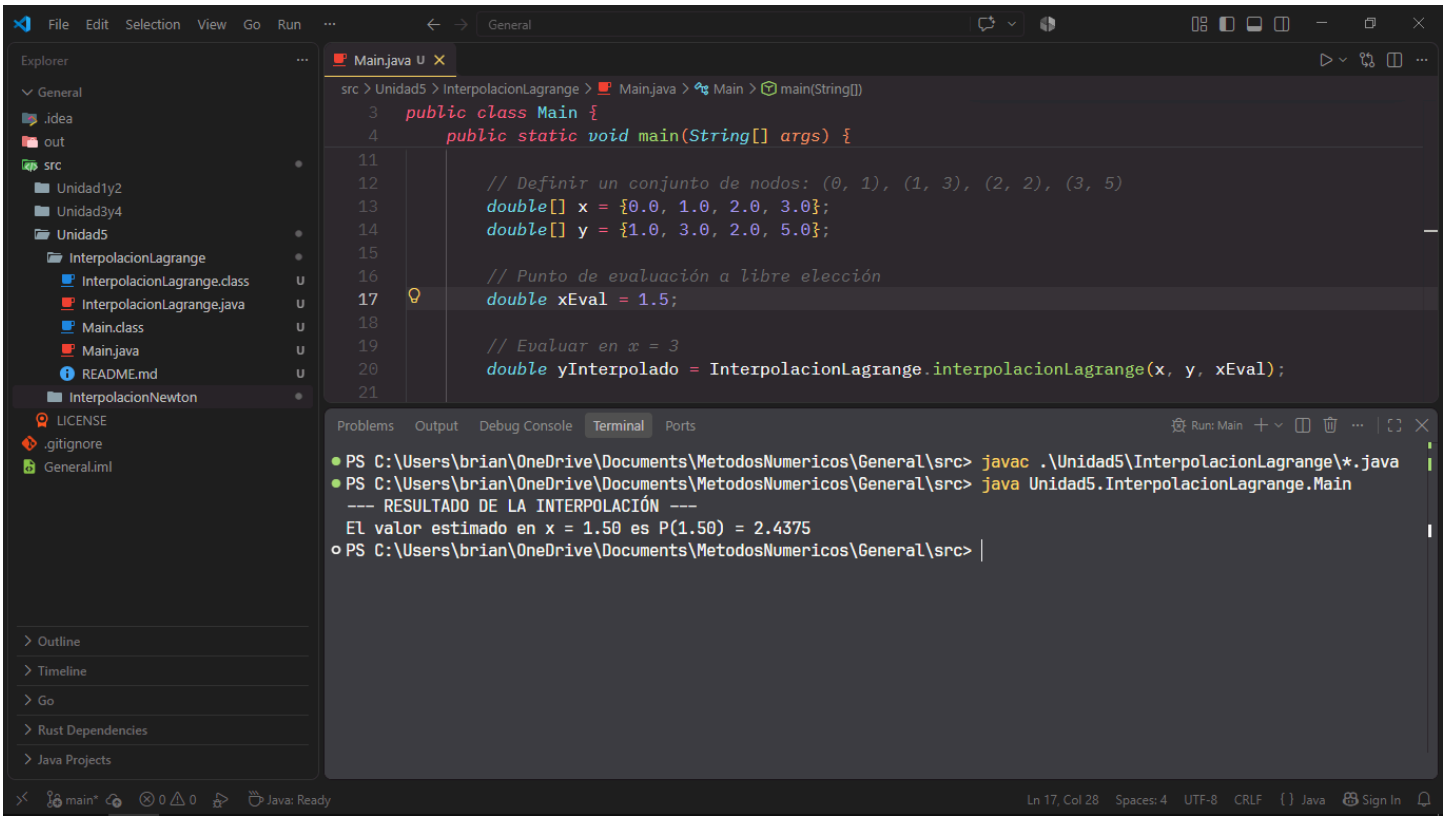
Conjunto de datos elegido

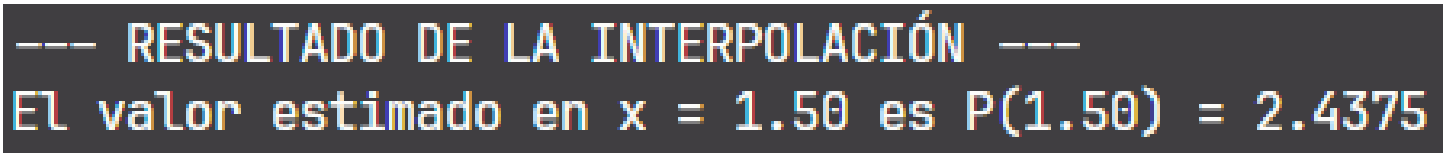
Se seleccionan cuatro nodos con el objetivo de mantener la claridad en la tabla de diferencias divididas:

i	x_i	y_i
0	0.0	1.0
1	1.0	3.0
2	2.0	2.0
3	3.0	5.0

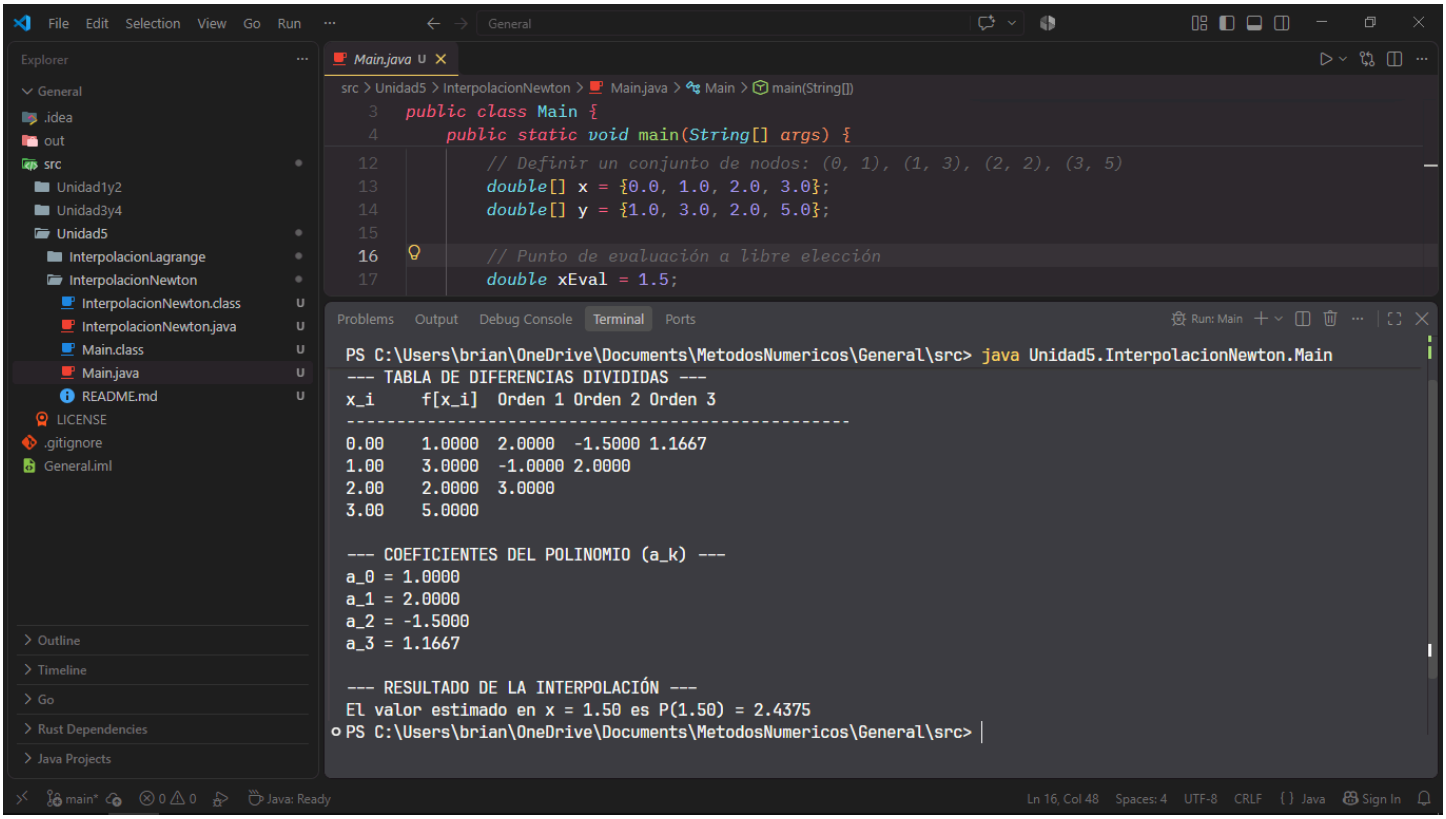
Punto de evaluación: x=1.5

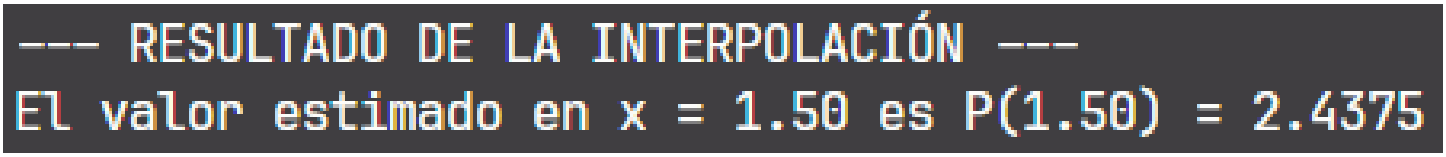
Para Lagrange:





Para Newton (incluyendo la matriz):





Análisis Comparativo y Preguntas de Discusión

¿Ambos métodos entregan exactamente el mismo resultado para el punto evaluado?

Sí, entregan exactamente el mismo resultado. Esto no es una coincidencia numérica, sino una consecuencia del **teorema de unicidad del polinomio interpolador**: dados n+1 nodos distintos, existe un único polinomio de grado ≤n que pasa por todos ellos. Lagrange y Newton son simplemente dos formas algebraicas distintas de expresar ese mismo polinomio.

Justificación numérica: Para x = 1.5, tanto la evaluación directa de la forma de Lagrange como la evaluación de la forma de Newton (con los coeficientes obtenidos de la tabla) producen P(1.5)=2.4375. La diferencia entre ambos resultados, en aritmética de punto flotante, es típicamente del orden del error de redondeo (aprox. 10^{-15}), lo cual confirma la equivalencia teórica.

Si se añade un nuevo punto al conjunto de datos iniciales, ¿cuál método exige un menor esfuerzo computacional?

El método de Diferencias Divididas de Newton exige un menor esfuerzo.

Razón: En la forma de Lagrange, cada polinomio base $L_k(x)$ depende de todos los nodos. Al añadir un nuevo nodo x_{n+1}, todos los productos que definen cada $L_k(x)$ cambian, por lo que es necesario recalcular los n+1 polinomios base desde cero y luego rehacer la combinación lineal completa. El costo de actualización es $O(n^2)$.

En cambio, en la forma de Newton, el polinomio tiene una estructura incremental:

$$P_{n+1}(x) = P_n(x) + f[x_0, \ldots, x_{n+1}] \prod_{j=0}^{n} (x - x_j)$$

Los coeficientes ya calculados $f[x_0]$, $f[x_0, x_1]$, …, $f[x_0, \ldots, x_n]$ permanecen invariantes. Solo se necesita calcular la nueva diferencia dividida $f[x_0,\ldots,x_{n+1}]$ (lo cual requiere O(n) operaciones adicionales) y añadir un término al polinomio. El costo de actualización es O(n), frente a $O(n^2)$ de Lagrange.

¿Qué complejidad algorítmica O(n) presenta la evaluación de cada algoritmo?

Algoritmo	Complejidad de construcción	Complejidad de evaluación en un punto
Lagrange	$O(n^2)$	$O(n^2)$
Newton	$O(n^2)$	$O(n)$ (con Horner)

Explicación:

- **Lagrange**: Para evaluar P(x) en un punto, se deben calcular los n+1 polinomios base $L_k(x)$. Cada $L_k(x)$ requiere un producto de n factores, y hay n+1 tales polinomios. Por tanto, la evaluación directa cuesta $O(n^2)$ operaciones .
- **Newton**: Una vez obtenidos los coeficientes (diferencias divididas), la evaluación puede realizarse mediante el esquema de Horner anidando los productos:

$$P(x) = f[x_0] + (x - x_0)\big(f[x_0, x_1] + (x - x_1)(\cdots)\big)$$

$$P_n(x) = a_0 + (x - x_0)\Big(a_1 + (x - x_1)\big(a_2 + \ldots + (x - x_{n-1})a_n\big)\Big)$$

  Esta evaluación requiere solo O(n) multiplicaciones y sumas.

Conclusiones

La práctica permitió comprobar que, aunque los métodos de Lagrange y Newton conducen al mismo polinomio interpolador, su comportamiento computacional difiere de manera significativa. Lagrange ofrece una formulación elegante y simétrica que facilita la comprensión teórica del problema de interpolación, pero su rigidez estructural lo hace poco adecuado para escenarios donde el conjunto de datos se actualiza con frecuencia. Newton, por su parte, demuestra que una elección inteligente de la base del espacio de polinomios puede traducirse en ventajas prácticas sustanciales: la actualización incremental y la evaluación eficiente mediante Horner lo convierten en la opción preferible en aplicaciones reales de análisis numérico.

La corrección del código de Lagrange reforzó un aspecto esencial de la programación científica: la fidelidad a la definición matemática. Un descuido aparentemente menor “como olvidar excluir el índice j=k” conduce a errores de división por cero o resultados indeterminados, lo que subraya la importancia de verificar cada estructura de control contra la fórmula que pretende implementar.

Referencias Bibliográficas

Burden, R. L., & Faires, J. D. (2011). *Numerical Analysis* (9.ª ed.). Brooks/Cole, Cengage Learning.

Chapra, S. C., & Canale, R. P. (2015). *Métodos Numéricos para Ingenieros* (7.ª ed.). McGraw-Hill Education.

Kincaid, D., & Cheney, W. (2009). *Numerical Mathematics and Computing* (6.ª ed.). Brooks/Cole, Cengage Learning.

Universidad Politécnica de Madrid. (2024). *Recurso T6: Interpolación por Newton* [Apuntes de clase]. Cooperativo LITI Digym. https://cooperativo.liti.digym.upm.es/wp-content/uploads/2024/12/RECURSO-T6-Apuntes-interpolacion-Newton.pdf

Universidad de La Laguna. (s.f.). *Algunos tópicos en teoría de aproximación* [Documento académico]. Repositorio Institucional RIULL. https://riull.ull.es/xmlui/bitstream/handle/915/33370/Algunos%20Topicos%20en%20Teoria%20de%20Aproximacion.pdf